

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1504

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I S. Jefna Princy- 192572058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy,

encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

REVERSE ENGINEERING TASK

Q1. Big Data Platform for Website Clicks, Mobile Interactions and Customer Transactions

Inferred System Architecture

The platform is most likely a **distributed big-data architecture** designed to collect and process billions of records generated from websites, mobile applications, and customer transactions.

A probable flow is:

Website / Mobile Apps / Transaction Systems



Data Ingestion Layer



Distributed Storage



Batch + Stream Processing



Analytics Layer



Dashboards / Business Intelligence

Distributed Storage System

The platform would probably use a distributed storage system such as a **distributed file system or cloud object storage**.

This is necessary because billions of records cannot efficiently be stored on a single server.

Structured data such as customer transactions can be stored in tables, while semi-structured data such as click events and application logs can be stored in formats such as JSON or Parquet.

Data can be partitioned based on **date, region, customer, or event type**. Partitioning reduces the amount of data that needs to be scanned during queries and therefore improves performance.

Data Ingestion Framework

The system would require both **batch ingestion and real-time ingestion**.

- Batch ingestion handles large volumes of historical transaction and customer data.
- Streaming ingestion continuously receives website clicks, mobile interactions, and application events.

A distributed messaging or event-streaming system would likely be used to handle the continuous flow of events.

Stream Processing

For real-time analytics, a **stream-processing engine** would probably process incoming events immediately.

It can identify patterns such as:

- sudden increases in website traffic
- customer activity
- transaction trends
- abnormal activity

- real-time business events

For large workloads, processing can be distributed across multiple nodes so that several parts of the data are processed simultaneously.

Scalability and Partitioning

The architecture would use **horizontal scaling**, where additional processing or storage nodes can be added when the workload increases.

Data partitioning allows different nodes to process different portions of the dataset simultaneously. Replication would also be used so that data remains available even if one storage node fails.

Analytics and Business Intelligence

After processing, cleaned and aggregated data would be transferred to the analytics layer. BI dashboards could display:

- customer activity
- sales trends
- website traffic
- transaction statistics
- regional performance

This allows business users to make decisions using both historical and real-time information.

Reverse-Engineering Justification

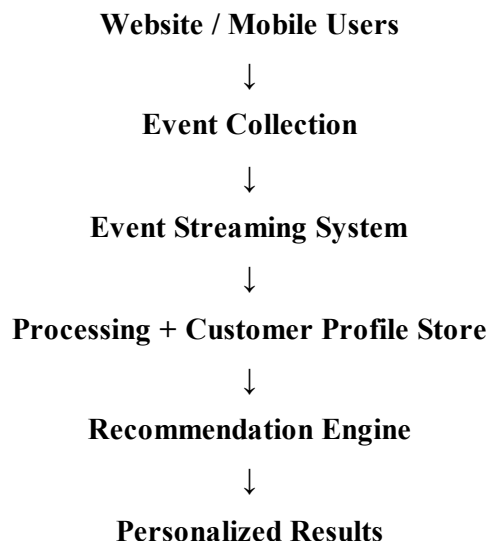
The requirement to process **billions of records** strongly suggests distributed storage and distributed processing rather than a traditional single-server database. The presence of both batch and real-time requirements indicates that the architecture likely combines **batch-processing and stream-processing pipelines**.

Q2. E-Commerce Platform with Real-Time Personalization

Inferred System Architecture

The e-commerce system is likely based on an **event-driven architecture**.

A probable flow is:



Event Streaming

Events such as:

- cart additions
- product views
- abandoned checkouts
- payments
- clicks

would probably be generated continuously by the website and mobile applications.

An event-streaming platform could collect these events and distribute them to different processing services.

Customer Profiling

The system would maintain customer profiles containing information such as browsing history, previous purchases, preferences, and interactions.

The profile data could be stored in a **NoSQL or distributed database**, allowing fast access for personalization.

Synchronization

Streaming events would update customer profiles continuously. For example, if a customer repeatedly views a particular product category, that activity can immediately update the customer's profile.

The recommendation system can then use the updated profile to generate relevant products.

Caching

A **distributed caching layer** would probably be used for frequently accessed information such as:

- product details
- customer preferences
- recommendations
- popular products

Caching reduces database access and provides faster responses to customers.

Structured and Semi-Structured Data

Transaction records are highly structured and could be stored in relational databases.

Browsing events and clickstream information are more flexible and could be stored in semi-structured formats or NoSQL systems.

These datasets can later be combined in the analytics pipeline.

Scalability and Fault Tolerance

The platform needs to handle sudden increases in traffic during sales and promotional events.

Therefore, services can be horizontally scaled by adding more application and processing nodes. Replication and distributed storage can prevent data loss when individual servers fail.

Machine Learning and Recommendations

Machine-learning models would likely analyze:

- previous purchases
- browsing behaviour

- product interactions
- customer preferences

to predict which products a customer is likely to be interested in.

The resulting recommendations can be sent back to the website in real time.

Dashboards and KPI Monitoring

Management dashboards could monitor:

- conversion rate
- abandoned carts
- sales
- product views
- recommendation performance
- regional activity

This provides operational and business visibility.

Reverse-Engineering Justification

The requirement for **real-time personalization** indicates that the system cannot depend only on traditional batch processing. Continuous event streaming, fast customer-profile access, caching, and machine-learning recommendation services are therefore probable components.

Q3. Big Data Healthcare System

Inferred System Architecture

The healthcare platform handles different types of sensitive information, including patient histories, diagnostic images, wearable sensor data, and prescription records.

A probable architecture is:

Hospitals / Laboratories / Wearable Devices



Data Integration Layer



Hybrid Data Storage



Stream + Batch Processing



Machine Learning / Analytics



Clinical Decision Support

Hybrid Database Architecture

A **hybrid storage architecture** would be suitable because healthcare data contains both structured and unstructured

information.

Structured data includes:

- patient records
- prescriptions
- laboratory results
- appointment information

Unstructured or large multimedia data includes:

- MRI images
- diagnostic images
- medical documents

Different storage technologies can therefore be used according to the characteristics of the data.

Data Integration

Healthcare organizations generate data from multiple independent systems. Therefore, data integration tools would likely connect:

- hospitals
- laboratories
- insurance systems
- wearable devices
- electronic health record systems

The integration layer would convert and standardize data before sending it to the analytics platform.

Streaming and Machine Learning

Wearable devices continuously generate sensor readings. A streaming pipeline can process this information in near real time.

Machine-learning models could identify patterns associated with health risks and support disease prediction or monitoring.

Historical patient information could be processed in batches to train and improve these models.

Privacy and Security

Healthcare information is highly sensitive, so strong security mechanisms would be expected.

The platform would likely use:

- encryption
- authentication
- authorization
- role-based access control
- audit logging

Different users should have different levels of access. For example, a doctor may access clinical records while an administrator may have access only to operational information.

Compliance

The system would need to follow applicable healthcare data-protection and privacy requirements. Data access should be logged so that organizations can determine who accessed or modified sensitive information.

Analytics Pipeline

The probable pipeline is:

Healthcare Data Sources

- **Integration**
- **Cleaning and Standardization**
- **Distributed Storage**
- **Processing**
- **ML Models**
- **Clinical Analytics**

The processed information can support disease prediction, patient monitoring, and population-health studies.

Reverse-Engineering Justification

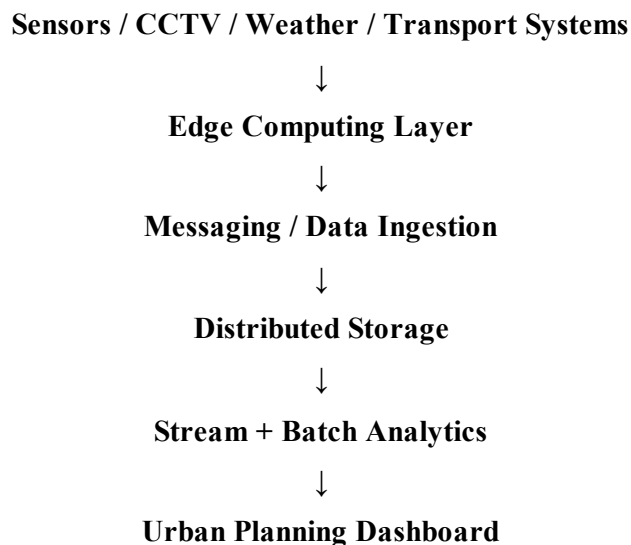
The combination of **structured records, diagnostic images, continuous wearable streams, and prescriptions** strongly indicates the need for hybrid storage and multiple processing approaches. The sensitivity of the information also makes encryption, access control, and auditing essential architectural components.

Q4. Smart City Platform

Inferred System Architecture

The smart-city system continuously receives information from traffic sensors, CCTV systems, weather services, and public transport systems.

A probable architecture is:



Edge Computing

Edge computing would likely be used near traffic sensors and CCTV systems.

Instead of sending every raw piece of data to a central cloud, initial processing can happen close to the data source. This reduces network traffic and provides faster responses for time-sensitive applications.

For example, traffic information can be processed locally to identify congestion before sending summarized information to the central platform.

Distributed Messaging

A distributed messaging system would collect continuous events from city infrastructure.

It can handle large numbers of messages from:

- traffic sensors
- public transport
- CCTV systems
- environmental sensors

This allows different analytics services to consume the same data.

Databases

Different databases may be required for different workloads.

A **time-series database** would be suitable for sensor readings collected over time.

A **geospatial database** would be useful for location-based information such as:

- traffic locations
- vehicle positions
- congestion areas
- transport routes

Stream and Batch Analytics

Stream analytics can identify current traffic congestion and transport problems.

Batch analytics can examine historical information to identify long-term patterns, such as recurring congestion during particular hours or days.

Combining both approaches provides both **real-time operational information and long-term planning insights**.

Security and Access Control

The platform would require:

- authentication
- authorization
- encrypted communication
- role-based access
- secure APIs

Different users such as traffic operators, city administrators, and analysts should receive access according to their responsibilities.

Visualization

Dashboards can provide live operational information using maps, charts, and alerts.

For example, a traffic-control team could see congestion locations and public-transport activity in real time.

Reverse-Engineering Justification

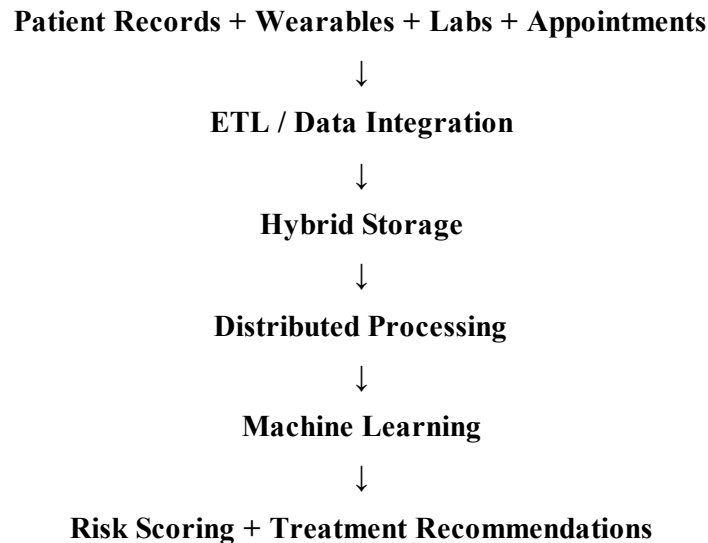
The continuous nature of traffic and sensor information suggests a **streaming architecture**, while the need for historical urban planning requires batch analytics. The presence of location-based queries makes time-series and geospatial data management probable components.

Q5. Healthcare Analytics System

Inferred System Architecture

This system combines patient records, wearable-device readings, laboratory reports, and appointment information.

A probable architecture is:



Hybrid Storage Architecture

Structured information such as patient records and appointments can be stored in relational databases.

Laboratory reports and other less-structured information can be stored using document or object-based storage.

Wearable-device readings require scalable storage because they can generate continuous streams of data.

ETL and Interoperability

ETL pipelines would extract healthcare information from different systems, transform it into a common format, and load it into the analytics environment.

Healthcare interoperability standards could help different hospitals and clinical systems exchange information consistently.

Real-Time Monitoring

Wearable devices continuously produce information such as health measurements.

A streaming-processing layer can analyze incoming readings and detect unusual patterns quickly.

This allows the system to provide alerts or trigger further analysis when required.

Predictive Health Analytics

Machine-learning models could analyze historical patient information together with wearable and laboratory data.

Possible outputs include:

- health-risk scores
- disease-risk predictions
- patient monitoring
- treatment-support recommendations

The models should support healthcare professionals rather than replace clinical decision-making.

Distributed Processing

Healthcare analytics can involve very large datasets. Distributed processing allows workloads to be divided among multiple machines, reducing processing time and allowing the system to scale as the number of patients and data sources increases.

Encryption and Compliance

Because healthcare information is sensitive, data should be encrypted during storage and transmission.

Access should be controlled using authentication and role-based authorization. Audit logs should record important access and modification activities.

Data governance policies would define how healthcare data is collected, stored, accessed, retained, and used.

Machine Learning for Risk Scoring

The ML pipeline would probably involve:

Data Collection → Cleaning → Feature Preparation → Model Training → Validation → Risk Prediction

Models could identify patterns associated with increased health risks and provide information that supports clinical decisions.

Reverse-Engineering Justification

The combination of **patient records, wearable data, laboratory reports, and appointment histories** indicates a hybrid data architecture with ETL and interoperability mechanisms. Continuous wearable information further suggests real-time processing, while predictive healthcare requirements justify the use of machine-learning pipelines.